You

I currently have a SharePoint flow that retrieves a file upload from a form and stores it as a string like the following: "[{"name":"New A & E Case 1 (1)_Akhtar Hasan.docx","link":"https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/_layouts/15/Doc.aspx?sourcedoc=%7B79406FD8-7571-41EE-AAE8-1AA56C9903B0%7D&file=New%20A%20%26%20E%20Case%201%20(1)_Akhtar%20Hasan.docx&action=default&mobileredirect=true","id":"01ZBAD6XGYN5AHS4LV5ZA2V2A2UVWJSA5Q","type":null,"size":26318,"referenceId":"01ZBAD6XFVCUXPHDA7UVCYFLJ2YOJBTKNH","driveId":"b!b-xtZOBWQ0Oe-XiEdHGghpF3bVoRZQlJgnAUAZTm-T4bglJpb7XbQaOd88bIWci0","status":1,"uploadSessionUrl":null}]" The file upload on the form is usually taken from the user's OneDrive. How can I upload this file to a SharePoint site where it can be stored for audit purposes?

Copilot

To upload the file from OneDrive to a SharePoint site for audit purposes, you can use the following approach:

1. **Extract File Details**: Parse the JSON string to extract the file's name and link. This will allow you to access the file in OneDrive.
2. **Download the File**: Use the file's link to download it from OneDrive. You can use an HTTP request action in your flow to achieve this.
3. **Upload to SharePoint**: Once the file is downloaded, use the "Create File" action in your SharePoint flow to upload the file to the desired SharePoint document library. You'll need to specify the SharePoint site URL, the document library, and the file content.

Here's a step-by-step guide for your flow:

- **Step 1: Parse JSON**
  Use the "Parse JSON" action in Power Automate to parse the string and extract the file details (e.g., name and link).
- **Step 2: HTTP Request to Download File**
  Use the "HTTP" action to send a GET request to the file's link. This will retrieve the file content.
- **Step 3: Create File in SharePoint**
  Use the "Create File" action to upload the file to your SharePoint site. Map the file content from the previous step to this action.

If you're new to Power Automate or need more details, you can find helpful resources on moving files between OneDrive and SharePoint or uploading files to SharePoint libraries.

Let me know if you'd like a more detailed walkthrough!

You

What schema should I use in Parse json if the string is like this: "[{"name":"New A & E Case 1 (1)_Akhtar Hasan.docx","link":"https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/_layouts/15/Doc.aspx?sourcedoc=%7B79406FD8-7571-41EE-AAE8-1AA56C9903B0%7D&file=New%20A%20%26%20E%20Case%201%20(1)_Akhtar%20Hasan.docx&action=default&mobileredirect=true","id":"01ZBAD6XGYN5AHS4LV5ZA2V2A2UVWJSA5Q","type":null,"size":26318,"referenceId":"01ZBAD6XFVCUXPHDA7UVCYFLJ2YOJBTKNH","driveId":"b!b-xtZOBWQ0Oe-XiEdHGghpF3bVoRZQlJgnAUAZTm-T4bglJpb7XbQaOd88bIWci0","status":1,"uploadSessionUrl":null}]"

Copilot

To use the "Parse JSON" action in Power Automate for the string you provided, you'll need to define a schema. Here's an appropriate schema based on the structure of your JSON:

```
{
  "type": "array",
```

```
  "items": {
    "type": "object",
    "properties": {
      "name": {
        "type": "string"
      },
      "link": {
        "type": "string"
      },
      "id": {
        "type": "string"
      },
      "type": {
        "type": ["string", "null"]
      },
      "size": {
        "type": "integer"
      },
      "referenceId": {
        "type": "string"
      },
      "driveId": {
        "type": "string"
      },
      "status": {
        "type": "integer"
      },
      "uploadSessionUrl": {
        "type": ["string", "null"]
      }
    },
    "required": ["name", "link", "id", "size", "referenceId", "driveId", "status"]
  }
}
```

Notes:

1. **Type Definitions**:
   - Fields like *type* and *uploadSessionUrl* are defined as *["string", "null"]* since they might be *null* in the JSON.
   - The *size* and *status* fields are defined as integers.
2. **Required Fields**:
   - I included fields that seem essential (*name*, *link*, *id*, etc.) in the *required* array. You can adjust this based on which fields you actually need.
3. **Structure**:
   - Since the input is an array of objects (indicated by square brackets), the schema starts with *"type": "array"*.

Once you've added this schema to the "Parse JSON" action, you can easily reference the extracted fields like *name*, *link*, etc., in subsequent steps of your flow.

Let me know if you'd like more help refining your flow!

You

Action 'Send_an_HTTP_request' failed: URI path is not a valid Graph endpoint, path is neither absolute nor relative or resource/object is not supported for this connector. Resources: me,users. Uri: https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/_layouts/15/Doc.aspx?sourcedoc=%7B79406FD8-7571-41EE-AAE8-1AA56C9903B0%7D&file=New%20A%20%26%20E%20Case%201%20(1)_Akhtar%20Hasan.docx&action=default&mobileredirect=true

Copilot

The error indicates that the URI you're trying to access is not compatible with the Microsoft Graph API or the connector you're using. This usually happens when the link is a direct URL to a SharePoint UI interface, rather than an appropriate Microsoft Graph API endpoint.

How to Resolve This

1. **Verify the Link**: The link you provided (*https://bradfordgovuk-my.sharepoint.com/...*) is a direct link for accessing the file through a web browser and not a valid Graph API endpoint.
2. **Use the File ID**: Instead of the direct link, you should extract the *id* value from your JSON (*01ZBAD6XGYN5AHS4LV5ZA2V2A2UVWJSA5Q*). This *id* can be used with Microsoft Graph's API to access the file directly.
3. **Build the Correct Graph API Endpoint**: Use the following Microsoft Graph API format to construct a valid URI:
4. https://graph.microsoft.com/v1.0/drives/{driveId}/items/{itemId}/content
    - Replace {*driveId*} with the value from your JSON (*b!b-xtZOBWQ0Oe-XiEdHGghpF3bVoRZQlJgnAUAZTm-T4bglJpb7XbQaOd88bIWci0*).
    - Replace {*itemId*} with the *id* value (*01ZBAD6XGYN5AHS4LV5ZA2V2A2UVWJSA5Q*).

The resulting endpoint will look like this:

https://graph.microsoft.com/v1.0/drives/b!b-xtZOBWQ0Oe-XiEdHGghpF3bVoRZQlJgnAUAZTm-T4bglJpb7XbQaOd88bIWci0/items/01ZBAD6XGYN5AHS4LV5ZA2V2A2UVWJSA5Q/content

5. **Make an HTTP GET Request**: Use the constructed API URL with an HTTP GET request. Ensure that your HTTP action includes the required headers for authentication:
    - *Authorization: Bearer <your-access-token>*
6. **Upload the File to SharePoint**: Once you successfully retrieve the file content, you can use the "Create File" action to upload it to your SharePoint site.

Additional Notes:

- Ensure that your flow has the necessary permissions to access the Microsoft Graph API (e.g., Files.Read or Files.ReadWrite permissions for OneDrive files).
- If you encounter further issues with authentication or permissions, you might need to verify that your Azure Active Directory setup allows the necessary access.

Let me know if you'd like guidance on constructing the flow or troubleshooting further!

You

Action 'Send_an_HTTP_request' failed: URI path is not a valid Graph endpoint, path is neither absolute nor relative or resource/object is not supported for this connector. Resources: me,users. Uri: https://graph.microsoft.com/v1.0/drives/b!b-xtZOBWQ0Oe-XiEdHGghpF3bVoRZQlJgnAUAZTm-T4bglJpb7XbQaOd88bIWci0/items/01ZBAD6XHD3L5DDJUOS5BIGM3GXEIFGBDB/content

Copilot

This error typically indicates an issue with the URI or the permissions required to access the resource via the Microsoft Graph API. Let's troubleshoot step by step:

Potential Issues and Solutions

1. **Invalid or Expired Access Token**:
    - The access token used in the "Send an HTTP request" action might have expired or doesn't have the required permissions.
    - **Solution**: Ensure the flow generates a valid token with the correct permissions. For accessing files, you need at least *Files.Read*, *Files.ReadWrite*, or *Sites.ReadWrite.All*.
2. **Incorrect URI Format**:
    - Ensure the URI is structured correctly. For file content, the format should be:
      https://graph.microsoft.com/v1.0/drives/{driveId}/items/{itemId}/content

    - **Solution**: Double-check the driveId and itemId values from the JSON string. Confirm they match the resource you're trying to access. If there's a typo or mismatch, it can lead to this error.
3. **Graph API Permissions for "drives" and "items"**:
    - The service account or user triggering the flow must have proper permissions to access the OneDrive resource.
    - **Solution**: Verify the permissions of the account used in your flow. You can check this in the Azure Active Directory admin portal under "API Permissions" for your app registration.
4. **Resource Availability**:
    - The file or drive might have been moved, deleted, or isn't accessible by the account running the flow.
    - **Solution**: Manually verify that the file and drive ID exist and are accessible from your account.
5. **Connector or Service Limitation**:
    - Sometimes, the issue may arise from limitations in the Power Automate HTTP connector or restrictions on accessing certain resources via Graph API.
    - **Solution**: Check if you're using the appropriate connector to access OneDrive. If the HTTP action isn't working, consider trying the "OneDrive for Business" actions in Power Automate.

Testing and Debugging

- **Test the Endpoint**: Use a tool like Postman or Graph Explorer to send a GET request to the endpoint:
  https://graph.microsoft.com/v1.0/drives/b!b-xtZOBWQ0Oe-XiEdHGghpF3bVoRZQlJgnAUAZTm-T4bglJpb7XbQaOd88bIWci0/items/01ZBAD6XHD3L5DDJUOS5BIGM3GXEIFGBDB/content
  Include a valid Authorization header with your bearer token:
  Authorization: Bearer <access-token>
  If it works there, the issue is likely with the flow configuration.
- **Inspect Error Details**: Review the full error message in Power Automate, as it may provide more clues about the issue.

Let me know if you need more guidance on fixing this! I'd be happy to help you troubleshoot further.

You

How do I include a valid authorization header?

Copilot

To include a valid Authorization header in your HTTP request, you'll need to generate a bearer token and use it in the header. Here's how you can do this:

Steps to Include a Valid Authorization Header

1. **Register an App in Azure Active Directory (AAD):**
    - Go to the Azure portal.
    - Register a new application and make a note of the following details:
        - **Application (Client) ID**
        - **Directory (Tenant) ID**
        - **Client Secret** (create one under "Certificates & Secrets").
2. **Grant the Required Permissions:**
    - Under "API Permissions" for your app, add Microsoft Graph permissions such as:
        - *Files.Read* or *Files.ReadWrite* (to access file content)
        - *Sites.ReadWrite.All* (to access SharePoint).
    - Ensure that you **grant admin consent** for the permissions.
3. **Generate a Bearer Token:**
    - Use the *Client ID*, *Client Secret*, and *Tenant ID* to generate a token via the OAuth 2.0 endpoint:
      POST https://login.microsoftonline.com/{tenantID}/oauth2/v2.0/token
      Body Parameters:
        - *grant_type*: *client_credentials*
        - *client_id*: Your Application (Client) ID
        - *client_secret*: Your Client Secret
        - *scope*: *https://graph.microsoft.com/.default*
    - The response will include an *access_token*. Copy this token.
4. **Add the Authorization Header:** In your HTTP request, include the header as:
   Authorization: Bearer <access_token>

Example HTTP Request

Here's what the final request might look like:
Headers:
Authorization: Bearer eyJ... (your access token here)
Content-Type: application/json
Endpoint:
https://graph.microsoft.com/v1.0/drives/{driveId}/items/{itemId}/content
Method: *GET* to download the file or *PUT* to upload the file.

This process allows you to authenticate securely and access your resources via the Microsoft Graph API. If you're using Power Automate, you may need to handle this setup via a custom connector or other OAuth mechanisms.

Let me know if you'd like a more in-depth explanation of any of these steps!

You

I can't do any of this as I'm not an admin in my organisation, what other SIMPLE ways are there of using my file upload string to store the file in my SharePoint site
"[{"name":"New A & E Case 1 (1)_Akhtar Hasan.docx","link":"https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/_layouts/15/Doc.aspx?sourcedoc=%7B79406FD8-7571-41EE-AAE8-1AA56C9903B0%7D&file=New%20A%20%26%20E%20Case%201%20(1)_Akhtar%20Hasan.docx&action=default&mobileredirect=true","id":"01ZBAD6XGYN5AHS4LV5ZA2V2A2UVWJSA5Q","type":null,"size":26318,"referenceId":"01ZBAD6XFVCUXPHDA7UVCYFLJ2YOJBTKNH","driveId":"b!b-xtZOBWQ0Oe-XiEdHGghpF3bVoRZQlJgnAUAZTm-T4bglJpb7XbQaOd88bIWci0","status":1,"uploadSessionUrl":null}]"

Copilot

No worries, Hasan! Let's simplify this process and work with the tools and permissions you already have. Since admin access is out of reach, you can avoid the Graph API and use Power Automate's existing actions to handle this file. Here's a simpler approach:

Using Power Automate (Without Admin Privileges)

You can take advantage of the SharePoint and OneDrive connectors already available in Power Automate:

1. **Parse the File Upload String**:
   - Use the **"Parse JSON"** action with the schema I provided earlier to extract the file details like name and link.
2. **Download the File from the OneDrive Link**:
   - Use the **"HTTP"** action to download the file. The link in the JSON might not work directly, so:
     - You can try creating a temporary Power Automate step to open the link manually and download the file contents to confirm it is accessible.
     - Alternatively, use the OneDrive connector in Power Automate to find the file by its name or id and download it using the **"Get file content"** action.
3. **Upload the File to SharePoint**:
   - Use the **"Create File"** action from the SharePoint connector.
     - Specify the SharePoint site address.
     - Set the folder path where the file should be stored.
     - Map the file content from the previous step.

Quick Flow Example

1. **Trigger**: Start the flow when the file is uploaded via your form or another triggering event.
2. **Parse JSON**: Extract the relevant fields (name and link) from the string.
3. **Get File Content (OneDrive)**: Use the **"Get File Content Using Path"** or **"Get File Content"** action from OneDrive. Input the file path or ID to retrieve the file content.
4. **Create File (SharePoint)**: Use the extracted name and downloaded content to upload the file to a SharePoint library.

Important Notes

- If the direct link (link field) doesn't work in the HTTP action, this method allows you to leverage OneDrive's and SharePoint's native connectors without needing admin privileges.
- Ensure you have access to both the OneDrive file and the SharePoint library where you're storing the file.

Let me know how you'd like me to refine these steps! I'm here to help make this as painless as possible.

You

The problem is that the link:
"https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/_layouts/15/Doc.aspx?sourcedoc=%7B79406FD8-7571-41EE-AAE8-1AA56C9903B0%7D&file=New%20A%20%26%20E%20Case%201%20(1)_Akhtar%20Hasan.docx&action=default&mobileredirect=true"
rests neither in OneDrive or SharePoint but rather in between, how can I retrieve the actual link from the form?

Copilot

Ah, I see the issue now! The link provided is a web-based "view" link rather than a direct download or API-compatible link. These types of links are often generated when users access files stored in OneDrive or SharePoint via their browser, and they're not immediately usable for programmatic access.

To retrieve the actual file link or its content for processing, here's a simple and practical approach:

Approach 1: Use the Direct File ID

The JSON string includes both the id and driveId of the file. This means the file **does exist in OneDrive** or **SharePoint**, but the link provided is for browser access. You can use these IDs to retrieve the file programmatically.

Steps:

1. Use OneDrive or SharePoint's **"Get File Content Using Path"** or **"Get File Content"** actions in Power Automate:
    - Input the driveId and id to locate the file.
    - Map these fields directly from your parsed JSON.
2. If you don't see these actions, try OneDrive's **"List Files in Folder"** action to identify the file and work with it.

Approach 2: Manually Trace the File Location

If programmatic access isn't feasible:

1. Open the link in a browser.
2. Verify where the file is stored (OneDrive or SharePoint) by checking the URL for context clues like:
    - **/personal/**: Indicates it's likely in the user's OneDrive.
    - **/sites/** or **/teams/**: Indicates it's likely in a SharePoint site.

Once you determine the file's storage location, you can retrieve its direct link or use a native connector to access it.

Approach 3: Work Directly with the Form Submission

If the form submission itself is generating this "in-between" link:

1. Check how the form is configured to handle file uploads.
2. Adjust the form settings to directly store the files in a SharePoint document library or OneDrive folder rather than generating a browser link. Many tools (e.g., Microsoft Forms or third-party apps) allow this.

Would you like detailed guidance on adjusting the form settings or modifying your Power Automate flow based on one of these approaches? Let me know how you'd like to proceed!

You

It is stored in personal OneDrive, but the link is a SharePoint link:
"https://bradfordgovuk-my.sharepoint.com/personal/akhtar_hasan_bradfordcft_org_uk/_layouts/15/Doc.aspx?sourcedoc=%7B79406FD8-7571-41EE-AAE8-1AA56C9903B0%7D&file=New%20A%20%26%20E%20Case%201%20(1)_Akhtar%20Hasan.docx&action=default&mobileredirect=true"

Copilot

Ah, I see the challenge clearly now! Even though the file resides in your personal OneDrive, the link you've been provided is a SharePoint-style view link for file access. This is common when files are shared or integrated into organizational environments, making it tricky to programmatically retrieve the file directly.

Here's the simplified approach you can use to work with this:

Simpler Approach Using Power Automate Connectors

You can leverage the OneDrive for Business connector in Power Automate to retrieve the actual file content, bypassing the complexities of the SharePoint view link.

1. Extract the File Name
- Parse the JSON string to extract the *name* field (*New A & E Case 1 (1)_Akhtar Hasan.docx* in your example).

2. Locate the File in OneDrive
- Use the **"List Files in Folder"** action from the OneDrive for Business connector to list all files in the folder where your form uploads files.
- Search for the file using its *name* value.

3. Retrieve the File Content
- Once the file is identified, use the **"Get File Content"** action from the OneDrive for Business connector to retrieve its content.

4. Upload the File to SharePoint
- Use the **"Create File"** action from the SharePoint connector to upload the file to your desired SharePoint library. Map the file content and name to the appropriate fields.

Important Notes

- If your organization's OneDrive structure is standardized, you might already know the exact folder where files are stored, saving you the need to use the "List Files in Folder" action.
- Ensure you have access to both the source OneDrive and the target SharePoint library for this to work smoothly.

Would you like step-by-step guidance on setting up this flow in Power Automate? Let me know if you'd like me to assist further!

You

The problem is that the file could be uploaded from anywhere within a user's OneDrive, how can I search through the OneDrive of the user for the file (and not the owner, although the owner and user could be the same when testing)

Copilot

To search through the OneDrive of the user (not the owner) and locate the uploaded file programmatically, you can take advantage of Power Automate and the OneDrive connector. The good news is that OneDrive supports searching for files based on their name or properties.

Here's how to set it up:

Steps to Search the User's OneDrive for the File

1. **Extract the File Name**: Use the **"Parse JSON"** action to extract the *name* field from the file upload string. For example, extract:
   New A & E Case 1 (1)_Akhtar Hasan.docx
2. **Use OneDrive's "Search" Action**:
   - Power Automate provides a **"Search for Files"** action in the OneDrive for Business connector.
   - Set the **Search Query** to the file name extracted from the JSON (*New A & E Case 1 (1)_Akhtar Hasan.docx*).
   - This action will search through the user's entire OneDrive, regardless of the folder structure.
3. **Get File Content**:
   - Once the search result is returned, use the **"Get File Content"** action to retrieve the content of the file. Map the Id of the file returned by the search action to this step.
4. **Upload to SharePoint**:
   - Use the **"Create File"** action in the SharePoint connector to store the file in the desired SharePoint document library. Provide the library path and map the file content.

Handling Scenarios Where the File Isn't Found

If the file isn't located in the user's OneDrive:
- Use a condition in the flow to check the search results.
- Provide a fallback message or alternative action (e.g., notify the user or log the event).

Important Considerations

- **User's Access to Power Automate**: Ensure that the flow is being triggered from the user's account (or an account with delegated permissions to search their OneDrive).
- **Unique File Names**: If there's a chance of duplicate file names, you might need additional details (e.g., size or id) to identify the exact file.

This setup should allow you to handle the dynamic nature of user-uploaded files, even if the location within OneDrive varies. Let me know if you'd like help creating the flow!

You

In the OneDrive for business connector, there is no "Search for Files", the connectors are the following:

When a file is created
Trigger
This operation triggers a flow when a new file is created in a folder. Files larger than 50 MB will be skipped and not returned by this trigger. Files moved within OneDrive are not considered new files.
When a file is modified
Trigger
This operation triggers a flow when a file is modified in a folder. Files larger than 50 MB will be skipped and not returned by this trigger.
Create file
Trigger
This operation creates a file.
For a selected file
Trigger
This trigger allows you to start a flow for a selected file in OneDrive for Business.
List files in folder
This operation gets the list of files and subfolders in a folder.
When a file is created (properties only)
Trigger This operation triggers a flow when a new file is created in a folder. Files moved within OneDrive are not considered new files.
When a file is modified (properties only)
Trigger
When a file is modified (properties only) This operation triggers a flow when a file is modified in a folder.

Convert file
This operation converts a file to another format. The list of supported conversions can be found at https://aka.ms/onedriveconversions
Convert file using path
This operation converts a file to another format using the path. The list of supported conversions can be found at https://aka.ms/onedriveconversions
Copy file
This operation copies a file within OneDrive.
Copy file using path
This operation copies a file within OneDrive by path.
Create share link This operation creates a share link for a file.
Create share link by path
This operation creates a share link for a file using the path.
Delete file
This operation deletes a file.
Extract archive to folder
This operation extracts an archive file into a folder (example: .zip). Maximum archive size is 50 MB and 100 files inside.
Find files in folder
This operation finds files within a folder using search or name pattern match.
Find files in folder by path
This operation finds files within a folder by path using search or name pattern match.
Get file content
This operation gets the content of a file.
Get file content using path
This operation gets the content of a file using the path.
Get file metadata
This operation gets the metadata for a file.
Get file metadata using path
This operation gets the metadata of a file using the path.
Get file thumbnail
This operation gets the thumbnail of a file. The thumbnail will only be valid for 6 hours.
List files in root folder
This operation gets the list of files and subfolders in the root folder.
Move or rename a file
This operation moves or renames a file.
Move or rename a file using path
Move or rename a file using path This operation moves or renames a file using the path.
Update file
This operation updates a file.
Upload file from URL

Which of these can I use?

Copilot

Thanks for clarifying that you're working within the available connectors! From the list you've provided, here's the best approach using what's available:

Steps to Search for and Handle the File

1. **Extract the File Name**: Use the **"Parse JSON"** action (as previously discussed) to extract the *name* field (e.g., *New A & E Case 1 (1)_Akhtar Hasan.doc*x) from the file upload string.
2. **Use "List Files in Root Folder"**:
    - Start with the **"List Files in Root Folder"** action to get a list of all files and subfolders in the root of the user's OneDrive.
    - If the file isn't found in the root folder, you may need to **iterate through subfolders** to locate it.
3. **Or Use "Find Files in Folder"**:

    - If you already know a parent folder where the file might be stored, you can use the **"Find Files in Folder"** action with a search query set to the file name.
    - **Wildcard Search**: Add parts of the file name (e.g., *New A & E Case*) if it's not an exact match. This improves flexibility.
4. **Retrieve File Content**:
    - Once you locate the file, use the **"Get File Content"** or **"Get File Content Using Path"** action to retrieve the file's content.
5. **Upload to SharePoint**:
    - Use the **"Create File"** action to upload the file to the desired SharePoint site and library. Pass the file name and content from the previous step to this action.

Example Flow with Available Actions

1. **Trigger**: Start the flow when needed (e.g., file upload or manually).
2. **Parse JSON**: Extract the file *name* and *link*.
3. **"Find Files in Folder"/"List Files in Root Folder"**: Use one of these actions to locate the file based on its name.
4. **Condition (Optional)**: Add a condition to handle cases where the file is not found. For instance, notify the user or log the event.
5. **Get File Content**: Retrieve the file's content once it's located.
6. **Create File (SharePoint)**: Upload the file to a specific SharePoint folder.

Key Actions from Your List

Here are the ones you'd primarily use:

- **Parse JSON**: Extract file details.
- **List Files in Root Folder**: Check the root directory for files.
- **Find Files in Folder**: Search within a specific folder using the file name.
- **Get File Content**: Retrieve the file content once found.
- **Create File** (SharePoint): Upload the file to the destination library.

This setup stays within the capabilities of Power Automate without requiring Graph API or admin privileges. Let me know if you'd like more detailed steps for a specific action!

You

How can I perform this step: If the file isn't found in the root folder, you may need to iterate through subfolders to locate it.

Copilot

Iterating through subfolders in Power Automate can be done using a combination of actions and a loop to go deeper into the folder hierarchy. Here's a step-by-step guide to set this up:

Steps to Iterate Through Subfolders in OneDrive

1. **List Files in Root Folder**:
    - Start with the **"List Files in Root Folder"** action to retrieve all files and folders in the root directory.
2. **Filter for Folders**:
    - Add a **"Filter Array"** action to filter out only the items that are folders. You can use the *isFolder* property (if available) or check for null/empty values in the *file* property. Example condition:
      item()['folder'] is not null
3. **Loop Through Folders**:
    - Use an **"Apply to Each"** loop to process each folder returned by the filter.
    - Inside the loop, add the **"List Files in Folder"** action to list the contents of the current folder.
4. **Search for the File in Each Subfolder**:

    - Add a condition inside the loop to check if the file with the given name exists in the folder. This can be done using another **"Filter Array"** action on the output of the "List Files in Folder" step.
5. **Handle Nested Subfolders** (Optional):
    - If subfolders themselves contain further subfolders, repeat the process:
        - For each folder found, call the "List Files in Folder" action again.
        - This can create a recursive process where each subfolder is checked.
6. **Get the File Content** (Once Found):
    - If a match is found, exit the loop and use the **"Get File Content"** action to retrieve the file's content.
7. **Upload to SharePoint**:
    - After retrieving the file content, proceed with the **"Create File"** action in the SharePoint connector to upload it to the desired location.

Important Notes

- **Limitations**: Power Automate doesn't natively support recursion (calling the same flow repeatedly for each folder), so if the folder hierarchy is deeply nested, you'll need a robust design.
- **Exit Conditions**: Use conditions to stop further iterations once the file is found, saving unnecessary processing time.

Simplified Example Flow

1. **List Files in Root Folder** → Retrieve all files and folders in the root.
2. **Filter Array** → Keep only folders.
3. **Apply to Each** → Loop through each folder:
    - **List Files in Folder** → Get contents.
    - **Filter Array** → Check for the target file name.
    - **Condition** → If file is found:
        - Exit the loop.
        - Use "Get File Content" to retrieve it.

This allows you to dynamically search through subfolders until the file is located. Let me know if you'd like help setting up the specific details of this flow!

You

My first for each loop gives the following error:
The input parameter(s) of operation 'For_each_1' contains invalid expression(s). Fix invalid expression(s) for the input parameter(s) of operation 'For_each_1'.
The parameter I'm using is the body returned from List files in root folder

Copilot

The error you're encountering usually occurs when the input expression for the "For Each" loop is not correctly referencing the output from the previous step (in your case, the "List Files in Root Folder" action). Let's fix it step by step:

Steps to Resolve the Issue

1. **Verify the Output from "List Files in Root Folder"**:
    - Check the output of the **"List Files in Root Folder"** action by examining the **"body"** in the dynamic content or by running your flow and looking at the "Run History."
    - The "List Files in Root Folder" action should return an array of files and folders.
2. **Use the Correct Expression in "For Each"**:
    - Make sure that you are referencing the proper field in the "List Files in Root Folder" action.
    - For example, if the output body is an array, you should use something like:
      body('List_files_in_root_folder')['value']
    - This expression points to the value array where the list of files and folders is stored. Replace *List_files_in_root_folder* with the actual name of your action (use underscores for spaces).
3. **Assign the Output Explicitly**:
    - If dynamic content isn't working, you can explicitly assign the array from the "List Files in Root Folder" step by writing the correct expression (as above).
4. **Check the "For Each" Settings**:
    - Ensure that the "For Each" loop is set to iterate over the array, not a single object or a non-array value.

Tips for Debugging

- **Run the Flow Once**:
  - If possible, test the flow and look at the "Run History" to examine the structure of the "List Files in Root Folder" output. This will help you confirm the correct expression to use.
- **Log the Output**:
  - Insert a **"Compose"** action after the "List Files in Root Folder" action and map the body to it. This lets you see the structure of the data returned and helps you write the correct expression.

Common Pitfalls

1. **Empty Folder**:
   - If the root folder is empty, the "List Files in Root Folder" action will not return any items. Ensure there are files or folders to iterate over.
2. **Incorrect JSON Path**:
   - Make sure you are correctly accessing the array from the JSON response. Even a small typo can cause issues.

Would you like me to assist with the exact expression or help analyze an output sample if you provide one? Let me know how I can help you move forward!

You

Now it's saying the error in my condition:
The input parameter(s) of operation 'Condition' contains invalid expression(s). Fix invalid expression(s) for the input parameter(s) of operation 'Condition'.

Copilot

This error typically occurs when the expression within the condition block is either not referencing the right field or is improperly formatted. Let's go step by step to troubleshoot and resolve this:

Steps to Fix the Condition Block

1. **Verify What You're Comparing**:
   - Double-check what you are trying to evaluate in the condition block.
   - For example, are you checking if the file name matches a specific value, or are you verifying if a certain property exists?
2. **Ensure You're Referencing the Correct Field**:
   - Review the output from the action before the condition (e.g., "List Files in Folder").
   - Use the **Dynamic Content** panel to ensure you're selecting the correct property. Alternatively, if writing an expression manually, use the correct JSON path. For example:
     items('Apply_to_each')['name']
   - Replace *Apply_to_each* with the exact name of your loop.
3. **Validate Your Expression**:
   - Make sure your condition uses a valid expression format. A common example is:
     equals(items('Apply_to_each')['name'], 'New A & E Case 1 (1)_Akhtar Hasan.docx')
   - This checks if the *name* field equals the specific file name.
4. **Check the Data Type**:
   - Ensure you're comparing values of the same data type (e.g., string to string). If needed, use expressions like string() to convert values.

Simplified Example
If you're checking if the file name matches, the condition should look like this:

- **Left Value**: *items('Apply_to_each')['name']* (or select *name* from Dynamic Content).
- **Operator**: *equals*
- **Right Value**: *New A & E Case 1 (1)_Akhtar Hasan.docx* (or any specific value you're comparing).

Debugging Tips

1. **Log Your Data**:
   - Add a **"Compose"** action just before the condition and map the value you're testing. This lets you see exactly what data is being passed into the condition.
2. **Run the Flow**:
   - Check the "Run History" to ensure the input value for the condition is what you expect.

If you share the exact condition you're working with or the action data, I can guide you more precisely. Let's get this sorted!

You

The condition I'm using is taking the name returned from the Find files in folder-copy connector (where the limit is set to 1) and checking if it's not equal to the file name from my parse json output

Copilot

The error in your condition likely arises due to a mismatch in data types or improper referencing of dynamic content. Let's walk through how to properly configure this condition:

Steps to Fix the Condition

1. **Verify the Output from "Find Files in Folder-Copy"**:
   - Confirm the property returned from the connector that contains the file name. For example, if the connector output contains the file details, look for the specific name field.
   - You can validate this by adding a **"Compose"** action just after the "Find Files in Folder-Copy" connector and mapping the file name to check if it's returned as expected.
2. **Access the File Name**:
   - In the condition, reference the file name from "Find Files in Folder-Copy" like this:
     first(outputs('Find_files_in_folder-Copy')?['body']?['value'])?['name']
     Explanation:
     - *outputs('Find_files_in_folder-Copy')*: Fetches the connector output.
     - *['body']?['value']*: Accesses the *value* array where files are listed.
     - *first()*: Since the limit is set to 1, only the first item is relevant.
     - *['name']*: Extracts the name of the file.
3. **Reference the File Name from Parse JSON**:
   - Use the file name extracted from the **"Parse JSON"** step. Ensure you're referencing the correct name property.
4. **Configure the Condition**:
   - In the condition block, set the expression to compare the two values:
     not(equals(first(outputs('Find_files_in_folder-Copy')?['body']?['value'])?['name'], outputs('Parse_JSON')?['body']?['name']))
     Explanation:
     - *first(outputs('Find_files_in_folder-Copy')?['body']?['value'])?['name']*: Retrieves the file name from the "Find Files in Folder-Copy" step.
     - *outputs('Parse_JSON')?['body']?['name']*: Retrieves the file name from the "Parse JSON" output.
     - *not(equals(...)):* Checks if the two names are **not** equal.
5. **Debugging**:
   - If the error persists, add another **"Compose"** action to log both file names and confirm their values and data types.

Potential Pitfalls to Watch For

- **Empty Results**:
  - If "Find Files in Folder-Copy" doesn't return any files, first() will throw an error. Handle this scenario with a condition to check if the array is empty before accessing the file name.
- **Data Type Mismatch**:
  - Ensure both values being compared are strings. You can use the string() function to enforce this:
    string(first(outputs('Find_files_in_folder-Copy')?['body']?['value'])?['name'])

Let me know if you'd like me to refine these steps further or assist with a sample expression tailored to your flow!

You

What conditional check can I use to make sure that the result returned by "Find files in folder" is compatible with the Get File Content action

Copilot

To ensure that the result from **"Find Files in Folder"** is compatible with the **"Get File Content"** action, you'll need to confirm that the result contains the required *id* field. The *id* is the unique identifier needed by the **"Get File Content"** action to locate and retrieve the file.
Here's how you can set up a conditional check:

Conditional Check

You can use a condition in Power Automate to verify if the **"Find Files in Folder"** action returned a valid result.

1. **Expression to Check If a File is Found**: Use this expression in the condition to ensure that the result is not empty and contains an id:
   length(outputs('Find_files_in_folder')?['body']?['value']) > 0
   - *outputs('Find_files_in_folder')*: Refers to the "Find Files in Folder" action.
   - *['body']['value']*: Points to the array of files returned by the action.
   - *length() > 0*: Ensures the array is not empty.
2. **If the Condition is True**:
   - This means a file was found. You can then safely proceed to use the id field in the **"Get File Content"** action.
   - Use this dynamic reference to get the id of the first file:
     first(outputs('Find_files_in_folder')?['body']?['value'])?['id']
3. **If the Condition is False**:
   - Take an alternate action, such as logging an error, notifying the user, or stopping the flow.

Handling Errors for Compatibility

- If the "Find Files in Folder" action might return multiple files, consider limiting the result (which you've already done with limit = 1).
- Always validate that the file has the necessary properties, especially id. You can add an additional condition to check for this:
  not(empty(first(outputs('Find_files_in_folder')?['body']?['value'])?['id']))

Summary of Workflow

1. Use **"Find Files in Folder"** to locate the file.
2. Add a **Condition** to check:
   - *length(outputs('Find_files_in_folder')?['body']?['value']) > 0*
3. If true:
   - Proceed with **"Get File Content"** using the *id*.
4. If false:
   - Log a message or notify the user about the missing file.

With these checks, your flow will gracefully handle cases where the file is not found or doesn't have the required attributes for the next step. Let me know if you'd like more detailed help on how to implement this!

You

How can I prevent/catch this error before it happens:

The error indicates that the 'Find files in folder' action failed with a 404 status, meaning the resource could not be found. This could be caused by an incorrect or non-existent folder ID being used in the search. Did you verify that the folder ID provided is correct and that the folder exists in OneDrive?

These are my parameters for example:

```
{
  "query": "9c Business Manager - CEO - Job profile_Akhtar Hasan.doc",
  "id": "6e202211-2856-4d17-9ded-5beb8b8626b0",
  "findMode": "OneDriveSearch",
  "maxFileCount": 1
}
```

Copilot

To prevent or catch this 404 error before it happens in the **"Find Files in Folder"** action, you can add validation steps to ensure the folder exists and the parameters are correct. Here's how you can handle this issue:

Steps to Prevent or Catch the Error

1. Validate the Folder ID Before Searching:

- Use the **"Get Folder Metadata"** action from the OneDrive for Business connector before running the "Find Files in Folder" action. This action will confirm that the folder exists.
- Input the *id* from your parameters into the "Get Folder Metadata" action.
- If the folder doesn't exist or the *id* is invalid, handle this gracefully with a condition (see Step 3 below).

2. Test the Query Parameter:

- Double-check the *query* parameter. Ensure that it exactly matches the file name or part of the file name you're searching for.
- If the *query* is dynamic, confirm its value by logging it using a **"Compose"** action to avoid typos or mismatched names.

3. Use a Condition to Verify Folder Metadata:

- After the **"Get Folder Metadata"** action, add a condition to check if the folder exists. For example:
  not(empty(body('Get_Folder_Metadata')?['id']))
- If the condition is true (folder exists), proceed with the "Find Files in Folder" action.
- If the condition is false (folder doesn't exist), log an error or notify the user.

4. Set Default Fallback Behavior:

- If the *id* is incorrect or non-existent, you can implement a fallback step to handle this gracefully. For example:
  - Notify the user that the folder is missing.
  - Stop the flow with a message indicating what went wrong.

Example Workflow

1. **Trigger**: Start your flow.
2. **Get Folder Metadata**: Use the folder *id* parameter to verify the folder exists.
3. **Condition**:
   - **If the folder exists**: Proceed with the "Find Files in Folder" action.
   - **If the folder doesn't exist**: Log an error or notify the user.
4. **Find Files in Folder**: If the condition passes, perform the search with the provided parameters.
5. **Handle Errors in Find Files Action (Optional)**: Add a **"Configure Run After"** on the next step to catch errors in case the folder disappears or another issue occurs during execution.

Preventive Best Practices

- **Use Static or Verified Folder IDs**: If possible, store folder IDs in a secure location to avoid mismatches or errors due to manual entry.
- **Enable Retry Policy**: Configure retries in the "Find Files in Folder" action settings to handle transient issues like network latency or temporary unavailability.
- **Validate Input Dynamically**: Use a "Compose" or logging action to inspect dynamic inputs like *query* and *id* during the flow.

By validating the folder's existence before running the "Find Files in Folder" action, you'll eliminate the risk of encountering a 404 error. Let me know if you'd like help implementing this in your flow!

You

The folder is fine, it's just that the file is not present in that folder, how can I include the check for this without producing an error?

Copilot

If the folder is valid but the file might not exist within it, you can include a check in your flow to verify whether the file is present before proceeding, ensuring no errors are triggered. Here's how you can implement this:

Steps to Handle Missing Files Gracefully

1. **Run the "Find Files in Folder" Action**:
   - Use the **"Find Files in Folder"** action with your search query. Make sure it returns a list (even if empty) of files that match your query.
2. **Check if Any Files Were Found**:
   - Add a **Condition** action to verify whether the "Find Files in Folder" action returned any results.
   - Use the following expression in the **Condition** block to check if the results array is empty:
     length(outputs('Find_files_in_folder')?['body']?['value']) > 0
     - This ensures the condition passes only if the result contains at least one file.
3. **Handle Both Scenarios**:
   - **If True** (File is found):
     - Use the first file in the results array to proceed.
     - Extract the *id* field using:
       first(outputs('Find_files_in_folder')?['body']?['value'])?['id']
     - Pass this *id* to the **"Get File Content"** action.
   - **If False** (File is not found):
     - Add an alternate step, such as logging a message, notifying the user, or simply terminating the flow gracefully.
4. **Prevent Errors in "Get File Content"**:
   - The "Get File Content" action will now only run if the condition (file exists) is true, ensuring it doesn't throw an error for a missing file.

Summary of Workflow

1. **Find Files in Folder**: Search for the file using the query and folder ID.
2. **Condition to Check Results**:
   - Expression: *length(outputs('Find_files_in_folder')?['body']?['value']) > 0*
3. **If True**:
   - Proceed with "Get File Content" using the file's *id*.
4. **If False**:
   - Log the missing file or notify the user.

Optional Enhancement

You can add a **"Configure Run After"** step for downstream actions to handle potential errors more flexibly. For example, set downstream actions to run only if the "Find Files in Folder" action succeeds.

By adding this check, your flow will gracefully handle cases where the file is missing, avoiding unnecessary errors. Let me know if you'd like additional guidance!

Conversation by Fifa Utuber with Copilot on Tuesday, Mar 4 2025